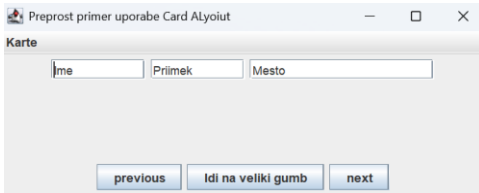


```

karte = new JPanel();
cl = new CardLayout();
karte.setLayout(cl);

```



```

addWindowListener(new WindowAdapter() {
    @Override
    public void windowIconified(WindowEvent e) {
        System.out.println(" anonimni inner posl
    }
    public void sploNiFajn() {}
});

```

```

JMenuItem m1 = new JMenuItem("Pregled");
m1.addActionListener(e->{cl.show(karte, "pregled");});

```

```

naslednji.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        cl.next(karte);
    }
});

```

```

prejsnji.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        cl.previous(karte);
    }
});

```

1

Dogodkovni model

- temelji na delegiranju
- **objekt poslušalec** za določeno vrsto dogodka je **primerek razreda**, ki implementira ustrezni vmesnik
 - za `ActionEvent` -e vmesnik `ActionListener`
 - za `KeyEvent` -e vmesnik `KeyListener`
 - za `WindowEvent` -e vmesnik `WindowListener`
 - ...

Lahko je primerek / instanca:

- samostojnega (neodvisnega) razreda
- notranjega razreda
- anonimnega notranjega razreda
- ... `this`

2

Adapter(ji)

- implementira(jo) ustrezni vmesnik
- vse metode imajo prazno telo
- so abstraktni (čeprav so vse metode implementirane)
- namesto, da sami implementiramo vse metode, dedujemo od ustreznega adapterja ter zagotovimo zgolj implementacijo posameznih metod
- ali obstajajo adapterji za vmesnike, ki imajo eno samo metodo?

Adapterski razredi:

```
java.awt.event.ComponentAdapter
java.awt.event.ContainerAdapter
java.awt.event.FocusAdapter
java.awt.event.KeyAdapter
java.awt.event.MouseAdapter
java.awt.event.MouseMotionAdapter
java.awt.event.WindowAdapter
```

3

```
import java.util.ArrayList;

public class Hisa {
    private String naziv;
    private ArrayList<Soba> sobe = new ArrayList<Hisa.Soba>();

    static class Nadstresnica {
        @Override
        public String toString() {
            return "Nadstresnica";
        }
    }

    class Soba {
        private String ime;

        Soba(String ime) {
            this.ime = ime;
            sobe.add(this);
        }

        @Override
        public String toString() {
            return "Sem " + ime + " v hisi " + naziv;
        }
    }

    public Hisa(String naziv) {
        this.naziv = naziv;
    }

    @Override
    public String toString() {
        return sobe.toString();
    }
}

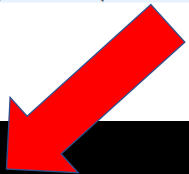
Hisa vila = new Hisa(" Vila Kamnica");
Hisa.Soba spalnica = vila.new Soba("spalnica");
Hisa.Soba otroška = vila.new Soba("otroška");

Hisa.Soba naVikendu = new Hisa(" Vikend na Pohorju").new Soba("Dnevna s kaminom");

new Hisa.Nadstresnica();
```

Compiled from "Hisa.java"
 class Notranji.Hisa\$Soba {
 final Notranji.Hisa this\$0;
 Notranji.Hisa\$Soba(Notranji.Hisa, java.lang.String);
 public java.lang.String toString();
 }

Compiled from "Hisa.java"
 class Notranji.Hisa\$Nadstresnica {
 Notranji.Hisa\$Nadstresnica();
 public java.lang.String toString();
 }



4

Pozor!

Primerki notranjih (inner) razredov lahko **dostopajo do atributov in metod zunanjega objekta** (primerka outer razreda), ta pa do vsega, kar **imajo gnezdeni / vsebovani objekti**.

Pri **statičnih** notranjih razredih **veljajo** identične **omejitve** kot smo že vajeni - static lahko namreč uporablja le statične metode in attribute zunanjega razreda.

5

Obravnava množic objektov

Collection Framework
(od 1.2)

(naziv container uporabljen v druge namene....)

6

Zakaj

- V aplikaciji nenehno oblikujemo nove objekte
- Koliko jih bomo imeli dejansko odvisno od izvajanja in zato šele takrat tudi znano
- Ena sama referenca (handle) zato ne zadostuje
- Polja?
- Polja so sicer "čistokrvni" objekti
- Polja so nefleksibilna, a zelo učinkovita, tip znan
- Preprosto linearno zaporedje elementov
- Ob kreiranju je potrebno določiti velikost, ki je nato ne moremo povečevati (length predstavlja velikost polja)

7

Polje je objekt!

```
int[] x;    //null referenca oz. handle
int[] y = new int[5];
int[] z = new int[4];

for (int i=0; i<y.length;i++)
    y[i]=i*i;

int[] w = {87,44,66};

x = new int[] {1,2};
```

8

Polje je objekt!

```
class Nekaj{...}

Nekaj[] polje;    //null referenca oz. handle
Nekaj[] p = new Nekaj[5];    //null reference
for (int i=0; i<p.length;i++)
    p[i]=new Nekaj();

Nekaj[] pol = {
    new Nekaj(), new Nekaj(), new Nekaj()
};

polje = new Nekaj [] { new Nekaj(), new Nekaj()};
```

9

Kaj pa če...

- Ker ne morem povečevati polja:
 - Kreiranje novega – večjega
 - Prenos vseh referenc v novo polje

- Prav to počel / počne npr. **Vector**

Vse elemente je obravnaval kot tip Object!!!

Torej smo lahko v nek vektor spravili
objekte različnih tipov!

10

Collection Framework

- v paketu **java.util**
(poleg starih collection razredov, pripomočki za delo z datumi in časom, internacionalizacijo ter drugih "utility" razredov kot so: bitno polje, Tokenizer nizov, generator naključnih števil)
- **Vector, Hashtable** (od 1.0)
- V polju so lahko reference ali osnovne vrednosti
- V **Collection**-ih pa le reference
(za osnovne tipe uporabimo ovojnice)
- **Od 1.5 Generiki**
(prej smo izgubili informacije o tipu!!!)

11

Collection Framework

- težave s polji

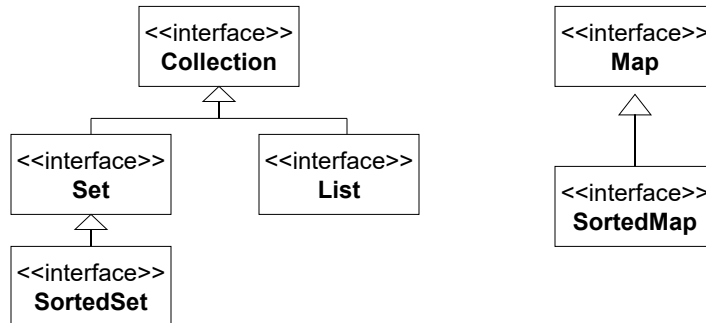
java.util.*

- **vmesniki**: List, Set, Map
SortedList, SortedMap, SortedSet
- **razredi**
ArrayList, TreeList, HashMap, TreeSet, TreeMap, LinkedList, HashSet
(Vector, Hashtable)
- **razred** Collections
sort(), shuffle(), reverse(),...

12

Ogrodje za delo z množicami objektov

- Skupino objektov obravnavamo kot enoto.
- Osnovni vmesniki za posamezne tipe množic:



13

Osnovne značilnosti

- `Collection`
ni posebnega zaporedja, dovoljuje duplikate
- `List`
urejenost oz. zaporedje, dovoljuje duplikate
- `Set`
ni posebnega zaporedja, ne dovoljuje duplikatov
- `Map`
podpira iskanje po ključu, vrednost katerega mora biti enolična

14

Vrste množic in namen

Collection	Korenski vmesnik (ne uporabljamo neposredno).
Set	Množica različnih objektov.
SortedSet	Urejena množica različnih objektov.
List	Seznam objektov
Map	Omogoča preslikavo ključa v vrednost.
SortedMap	Omogoča preslikavo urejenih ključev v vrednost.

15

List vs. Map

vmesnik List

add(Object)
get(index)

```
ArrayList<Nogometaš> nkMaribor = new ArrayList<Nogometaš>();
Nogometaš tava = new Nogometaš("Tavares", 176, 41, 36);
Nogometaš handa = new Nogometaš("Handanović", 174, 4, 11);
nkMaribor.add(tava);
nkMaribor.add(handa);
```

vmesnik Map

put(ključ, vrednost)
get(ključ)

```
HashMap<Integer, Nogometaš> ekipa = new HashMap<Integer, Nogometaš>();
ekipa.put(33, handa);
ekipa.put((Integer)10, zzTop);
ekipa.put(new Integer(2), miki);
ekipa.put(9, tava);
ekipa.put(11, zaho);
System.out.println(ekipa.get(9));
```

16

Parametrizirani tudi vmesniki

```

Iterator<Oseba> it = punce.iterator();
while (it.hasNext()) System.out.println(it.next());

public class PrimerjalsnikCevljev implements
    Comparator<Oseba>{

    @Override
    public int compare(Oseba prva, Oseba druga)

```

17

Nekateri razredi, ki implementirajo te vmesnike

Podatkovna struktura	Set	SortedSet	List	Map	SortedMap
Sekljalna tabela	HashSet			Hashtable HashMap	
Dinamično polje	ArrayList Vector				
Uravnoreženo drevo	TreeSet			TreeMap	
Povezani seznam	LinkedList				

18

Urejenost

```
java.lang.Comparable <Tip>
    compareTo(drugiObjekt)
```

```
java.util.Comparator <Tip>
    compare(prvi, drugi)
```

```
java.text.Collator
    Collator.getInstance(Locale.getDefault())
    implementira vmesnik Comparator
```

19

Lokalna pravila

```
imena.add("Janez");
imena.add("Ötzi");
imena.add("Đuro");

System.out.println(imena);

System.out.println("---- sortirani SLO ---");
Collections.sort(imena,Collator.getInstance(Locale.getDefault()));
// ali pa kar Collections.sort(imena,Collator.getInstance())
System.out.println(imena);

System.out.println("---- sortirani CRO ---");
Collections.sort(imena,Collator.getInstance(new Locale("hr","HR")));
System.out.println(imena);

System.out.println("---- sortirani D ---");
Collections.sort(imena,Collator.getInstance(Locale.GERMAN));
System.out.println(imena);

---- sortirani SLO ---
[Čvetka, Janez, Janez, Karmen, Lucija, Ötzi, Špela, Vid, Žarko, Žiga, Đuro]
---- sortirani CRO ---
[Čvetka, Đuro, Janez, Janez, Karmen, Lucija, Ötzi, Špela, Vid, Žarko, Žiga]
---- sortirani D ---
[Čvetka, Janez, Janez, Karmen, Lucija, Ötzi, Špela, Vid, Žarko, Žiga, Đuro]
```

20

Operacije nad seznamami

- **Razred** `java.util.Collections`

- `static void sort(List list)`
uredi seznam po privzetih pravilih
- `static void sort(List list, Comparator c)`
uredi seznam po pravilih „komparatorja“

```
Collections.sort(nkMaribor, new PrimerjalnikStarosti());
```

- `static void shuffle(List list)`
naključno permutira seznam objektov

21

Parametrizirani tudi vmesniki

```
import java.util.Comparator;

public class PrimerjalnikStarosti implements Comparator<Nogometaš>{

    @Override
    public int compare(Nogometaš o1, Nogometaš o2) {
        if (o1.getStarost()>o2.getStarost())
            return 1;
        else
            if (o1.getStarost()<o2.getStarost())
                return -1;
            else // sta enako mlada, uporabimo rezultat pravila privzete urejenosti
                return o1.compareTo(o2);
    }
}

public class Nogometaš implements Comparable<Nogometaš>{

    @Override
    public int compareTo(Nogometaš o) {
```

22

Operacije nad seznamami

- **Razred** `java.util.Collections`

- `static void sort(List list)`
uredi seznam po privzetih pravilih
- `static void sort(List list, Comparator c)`
uredi seznam po pravilih „komparatorja“

```
Collections.sort(nkMaribor, new PrimerjalnikStarosti());
```

- `static void shuffle(List list)`
naključno permutira seznam objektov

23

Parametrizirani tudi vmesniki

```
import java.util.Comparator;

public class PrimerjalnikStarosti implements Comparator<Nogometaš>{

    @Override
    public int compare(Nogometaš o1, Nogometaš o2) {
        if (o1.getStarost()>o2.getStarost())
            return 1;
        else
            if (o1.getStarost()<o2.getStarost())
                return -1;
            else // sta enako mlada, uporabimo rezultat pravila privzete urejenosti
                return o1.compareTo(o2);
    }
}

public class Nogometaš implements Comparable<Nogometaš>{

    @Override
    public int compareTo(Nogometaš o) {
```

24

Obdelava/"sprehod"

"foreach"

```
for (Oseba x:delavci) {
    x

}

Iterator it = delavci.iterator();
while (it.hasNext()) {
    Oseba x = (Oseba)it.next();
}
```

25

Obdelava/sprehod

```
for (Nogometaš x : nkMaribor)
    System.out.println(x);

Iterator<Nogometaš> kaz = nkMaribor.iterator();
while (kaz.hasNext())
    System.out.println(kaz.next());
```

26

Ena referenca lahko predstavlja objekte različnih razredov!

```
Investicija inv = new Nepremicnina ();
```

```
inv = new VarcevalniRacun();
```

```
inv= new KriptoValuta();
```

```
Smrtonosen x = new Sosed();
```

```
Fakinski joze = new Sosed();
```

```
x = new Zmajcek();
```

27

Velja tudi za generike...

```
List<String> imena;
```

```
imena = new ArrayList<String>();
```

```
imena = new Vector<String>();
```

```
imena = new LinkedList<String>();
```

```
Map<Integer,String> kosarkasi;
```

```
kosarkasi = new HashMap<Integer,String>();
```

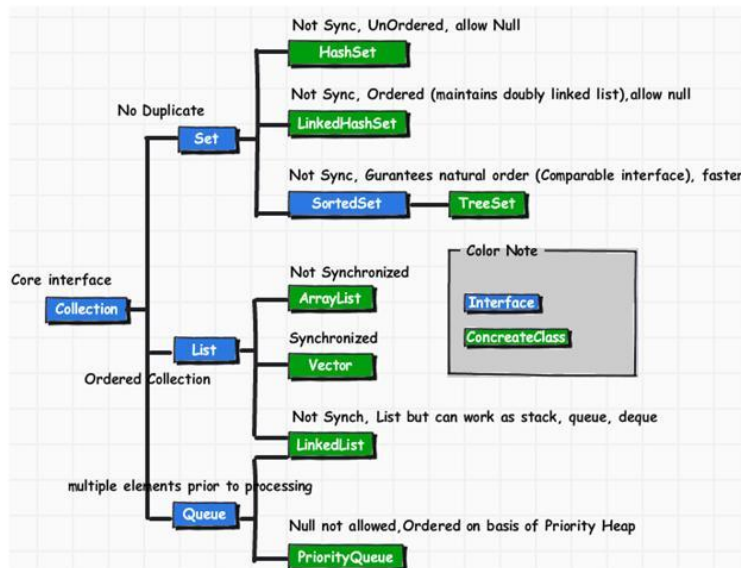
```
kosarkasi = new Hashtable<Integer,String>();
```

28

Logične operacije nad množico Set

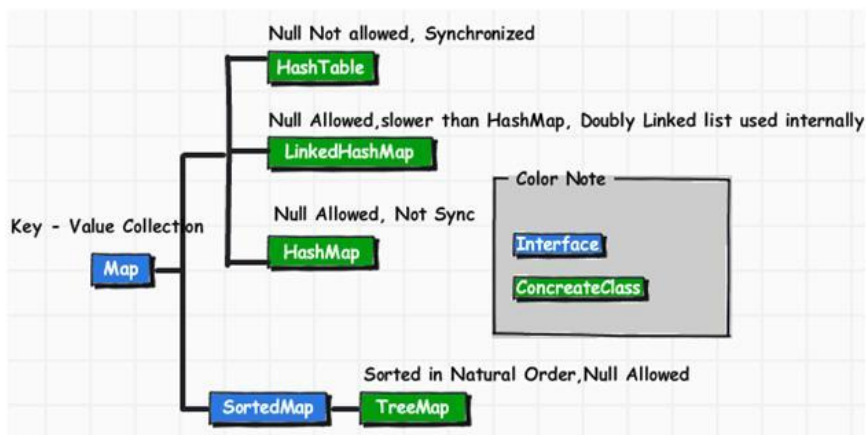
b podmnožica a	<code>a.containsAll(b)</code>
a = a unija b	<code>a.addAll(b)</code>
a = a razlika b	<code>a.removeAll(b)</code>
a = a presek b	<code>a.retainAll(b)</code>
a = prazna množica	<code>a.clear()</code>

29



<https://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>

30

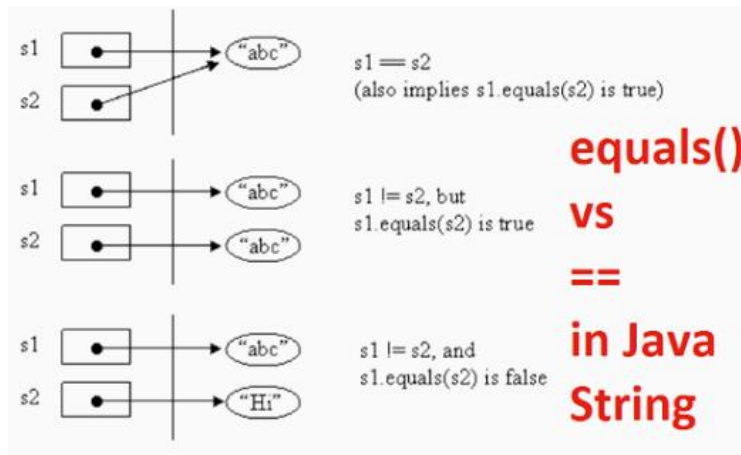


<https://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>

31

	HashSet	TreeSet
podatkovna struktura	hash tabela (interno uporablja HashMap)	Drevo (interno uporablja TreeMap)
Urejenost	neurejeno	po privzetih ali uporabniško definiranih pravilih (Comparator)
Primerjava in zaznava duplikatov	Uporablja metodi equals() in hashCode() za odstranjevanje morebitnih duplikatov	Za primerjavo uporablja metodo compareTo() ali metodo compare()
časovna kompleksnost dodajanja/brisanja/iskanja	v povprečju O(1)	O(log n)
zaporedje iteracije	arbitrarno/nepredvidljivo	urejeno/sortirano
null	dovoljen en	ni dovoljen
poraba pomnilnika	zmerna	relativno velika

32



33

equals

privzeto implementirano kot `==`

če zagotovimo svojo implementacijo,
ne pozabimo še na `hashCode()`

3.2. `equals()` Method With Object Types

Let's go back to our `Person` class. For the `equals()` method to work correctly, we need to override the method in the custom class by considering the fields contained in the class.

```
public class Person {
    // other fields and methods omitted

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (o == null || getClass() != o.getClass())
            return false;
        Person person = (Person) o;
        return age == person.age && Objects.equals(name, person.name);
    }
}
```

First of all, the `equals()` method returns `true` if the given value has the same reference, which is checked by the reference operator. If not, we start the equality test.

34

"Thread safe" kolekcije

